tidySTL: Exposing Implicit Semantics in STL Evaluation

Sota Sato[1] and Masaki Waga[2]

[1] National Institute of Advanced Industrial Science and Technology (AIST)
[2] Kyoto University

Abstract. Robust semantics of Signal Temporal Logic (STL) provides a verdict, optimization objective, and training feedback across monitoring, falsification, and control. Evaluating a finite sampled trace, however, requires decisions that neither the STL formula nor the trace fully specify, such as interpolation and end-of-trace behavior. We call this behavior *implicit semantics*, which can differ among tools and lead to different downstream consequences such as falsification outcomes. We present tidySTL, a diagnostic toolkit that makes such evaluator behavior explicit and exposes their computations through an inspectable DAG. Using a set of controlled cross-tool comparison cases, tidySTL identifies and localizes differences in implicit semantics among existing tools. Its architecture also serves as an extensible evaluator layer: representative variations require only localized backend changes, while the formula, signal, and public interface remain fixed.

1 Introduction

Signal Temporal Logic (STL) is a standard formalism for specifying temporal properties of real-valued signals [16,10]. Its quantitative *robustness* semantics measures how strongly a behavior satisfies or violates a specification, and the resulting margin underpins runtime monitoring [2], falsification [1], and learning-enabled design [20].

The robustness semantics is defined over an idealized continuous signal, yet a tool usually evaluates it on a finite trace—and a formula and trace do not determine a robustness value on their own. Each tool fixes the rest, silently; we call this *implicit semantics* and organize the choices in Table 1. These choices are not cosmetic: they can flip a tool's verdict and propagate downstream. A constructed falsification task shows a search succeeding reliably under one setting but much less reliably under another (Figure 3). Prior cross-tool studies reconciled such disagreements by post-hoc independent checking [9]; we instead make the responsible choice explicit and switchable before evaluation.

We present tidySTL,[3] a diagnostic toolkit that does exactly this. It places the implicit choices behind a backend boundary, so the same formula and signal can be evaluated under named, swappable backends while the parser, formula representation, and public interface stay fixed (Figure 1). Each backend exposes its

[3] Artifact repository: `https://github.com/midoriao/tidystl`.

choices as inspectable intermediate computations; compatibility backends emulate existing tools for cross-tool diagnosis, and the same separation makes the evaluator layer extensible (Table 3).

This paper makes the following contributions.

- We organize the implicit semantic choices in STL robustness evaluation, and show that they can materially affect acceptance, verdicts, rankings, and downstream falsification (§2 and §4.1).
- We present an architecture that represents these choices as explicit, swappable backends and makes their computations inspectable (§3).
- Building on it, we emulate Breach, RTAMT, and STLCG++ as compatibility backends and *mechanically localize* their discrepancies to the responsible implicit choice and formula node (Table 2 and §4.1).
- We show that the architecture provides an extensible evaluator layer: representative variations require localized backend changes while the formula, signal, and public interface remain fixed (§4.2).

2 Implicit Semantic Choices in STL Evaluation

We take the quantitative STL semantics of Donzé and Maler [8] as the theoretical reference for evaluation. We recall here only the definitions needed to delimit where implicit semantic choices arise in evaluation. For completeness, Appendix A records the standard full inductive definition and further scope distinctions.

Definition 1 (Signal). *A* signal *over a set of variables* $\mathbf{Var}$ *is a function* $\sigma\colon \mathbb{R}_{\geq 0} \to \mathbb{R}^{\mathbf{Var}}$.

Definition 2 (STL syntax). *The set* $\mathbf{Fml}$ *of STL formulas is defined as follows:*

$$\mathbf{Fml} \ni \varphi ::= \top \mid f(\boldsymbol{x}) \geq 0 \mid \neg\varphi \mid \varphi_1 \vee \varphi_2 \mid \varphi_1 \wedge \varphi_2 \mid \varphi_1 \, U_I \, \varphi_2 \mid \varphi_1 \, R_I \, \varphi_2 \mid \Diamond_I \varphi \mid \Box_I \varphi.$$

Here f is a function symbol applied to variables from $\mathbf{Var}$*, for example $f(x, y) = 4x - y - 3$, and I is a non-singular interval in $\mathbb{R}_{\geq 0}$, for example $[2, 5]$, $(1, 3]$, or $[0, \infty)$.*

Informally, the robust semantics [8,10] $\rho(\varphi, \sigma, t)$ replaces the Boolean verdict with a signed margin (e.g., $\rho(x - 3 \geq 0, \sigma, t) = \sigma_x(t) - 3$), which the connectives and temporal operators aggregate by min, max, inf, and sup (e.g., $\rho(\Box_{[0,2]}\,(x - 3 \geq 0), \sigma, t) = \inf_{t' \in [t,\, t+2]}(\sigma_x(t') - 3)$).

This reference semantics is defined over an idealized signal, but what an evaluator receives is a finite *timed state sequence* $(t_0, \boldsymbol{v}_0), \ldots, (t_n, \boldsymbol{v}_n)$ of timestamped samples. Reading this finite representation as a signal on $\mathbb{R}_{\geq 0}$ is left to the tool, and a formula and trace do not determine it on their own. More generally, we call an evaluator decision an *implicit semantic choice* when it is not specified by the STL formula or finite input, but is fixed by adopting the evaluator and changes acceptance or reported robustness. Table 1 organizes the choices we consider.

Table 1. Implicit semantic choices in robustness evaluation. Indented rows decompose finite-trace interpretation. A indicates an effect on input acceptance, and R indicates an effect on reported robustness. Appendix A further explores the scope of these choices.

Implicit choice	What it fixes; representative variants	Effect
Finite-trace interpretation	How finite samples are evaluated as a signal	A/R
Signal model	Values between samples; piecewise-linear, piecewise-constant, or discrete	R
Window alignment	Edges between samples; exact or snapped to samples	A/R
Boundary rule	Windows past the trace end; clamp or pessimistic (empty)	R
Terminal-step rule	Reported final robustness; as-is or extend penultimate	R
Atomic predicates	Scoring of accepted predicates; signed margin or distance to satisfaction set	R
Nonstandard predicates	Scoring of extensions such as equality	R
Coverage	Accepted formulas, predicates, intervals, and traces	A
Implementation-specific	Other evaluator-fixed behavior, such as finite sentinels for infinity	A/R

3 Design and Implementation

This section turns the implicit choices of §2 into inspectable implementation structure. tidySTL exposes robustness evaluation through a small interface, while placing choices that vary across evaluators behind a backend boundary.

3.1 Evaluation Interface

tidySTL has two explicit inputs: formula and signal. A *formula* is a typed syntax tree, parsed from the textual STL front end. A *signal* contains one or more named, batched value sequences over a shared time grid.

An optional third input, the *backend*, encapsulates the implicit choices and the execution strategy. Evaluation takes the form `robustness(formula, signal, backend)` and returns an array of robustness values over time:

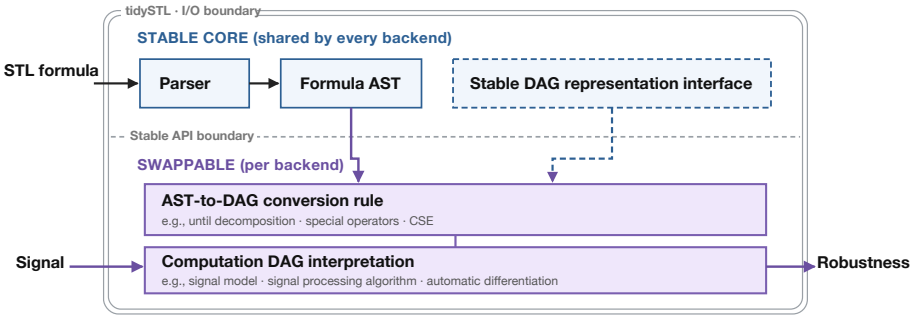

Fig. 1. Architecture of tidySTL. The stable core keeps formula and signal inputs identical across comparisons; swappable backend lowering and DAG interpretation make implicit choices explicit and their effects inspectable.

```python
from tidystl import Signal, parse, robustness
import numpy as np

phi = parse("G[0,5](F[0,2](velocity >= 10))")

times = np.linspace(0, 10, 200)
velocity_batch = np.array([np.sin(times), np.cos(times)])  # two traces
sig = Signal.from_dict(
    times=times,
    values={"velocity": velocity_batch},
)
rho = robustness(phi, sig)                    # default backend
rho = robustness(phi, sig, backend="breach")  # Breach-compatible
```

Keeping this interface small serves two purposes. For users, the same formula and signal can be evaluated under different documented backends without changing the surrounding application. For backend authors, new evaluation behavior attaches below a stable interface rather than requiring a new parser, signal representation, or calling convention.

3.2 Architecture

Internally, tidySTL separates a stable core from backend-specific evaluation logic, as shown in Figure 1. The stable core is shared by every backend, and it does not decide how (φ, σ) maps to a robustness value. Those decisions are delegated to the backend layer.

In the backend layer, the *implicit choices* fix what is computed: each backend resolves the implicit choices from Table 1 in a documented way. The *execution strategy* fixes how it is computed, without changing those choices.

The shared formula representation and explicit computation DAG provide *observability*: intermediate outputs from two backends can be aligned at shared

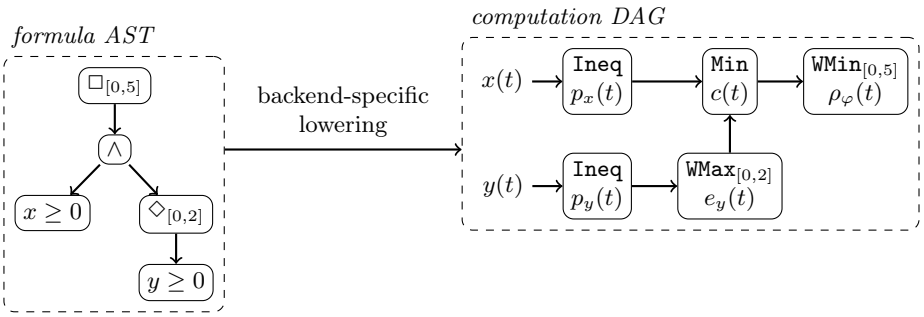

Fig. 2. Lowering of $\varphi :\equiv \square_{[0,5]}\big((x \geq 0) \wedge \diamond_{[0,2]}(y \geq 0)\big)$ from the formula AST (left) to a computation DAG (right). Nodes are typed primitive operations (`Min` is pointwise; `WMax`/`WMin` aggregate over the annotated window); edges carry the intermediate robustness signals, which remain inspectable in the evaluation result.

formula nodes to identify the minimal divergent node and its backend operations. §4.1 demonstrates this diagnostic workflow.

A backend realizes its choices in two steps: it lowers the formula AST into a computation *directed acyclic graph* (DAG) of typed primitive operations, and it interprets those operations into the chosen concrete signal-processing pipeline. The lowering fixes the implicit choices and the interpretation fixes the execution strategy, so semantics-preserving optimizations such as common subexpression elimination and caching of shared nodes stay on the executor side, never entangled with a semantic variant. Figure 2 illustrates the lowering on an example formula with nested temporal operators.

Extension points. A backend consists of primitive operations, a lowering rule, and an executor. Extensions may introduce new primitive operations during lowering or override the interpretation of existing ones. For programming details, the artifact repository gives the user manual (`docs/usage.md`) and reference implementations. In §4.2, we demonstrate that emulating an existing tool's behavior requires only localized changes to the backend with small effort. Despite this explicit structure, our performance sanity check found its overhead to be negligible (Appendix C).

Architectural byproducts. The swappable architecture (Figure 1) also supports capabilities outside the central scope of this paper. For example, it can accomodate advanced semantic extensions, such as smooth differentiable robustness [15,19], as well as performance-optimized backends, all through the same interface. The codebase also contains differentiable and SIMD-optimized backends as instances of this extensibility.

4 Evaluation

We evaluate tidySTL in two steps.[4] First, we show that implicit choices are not cosmetic: across tools, they can change verdicts and downstream falsification outcomes (§4.1). Second, we show that realizing such a choice in tidySTL is a localized change below the stable interface (§4.2).

4.1 Robustness Discrepancies Across Implicit Choices

This subsection first establishes, using real-tool runs, that the implicit choices listed in Table 1 produce observable cross-tool discrepancies. It then uses validated compatibility backends to localize the responsible choice and to illustrate downstream consequences.

Real-tool discrepancies. We first compare the original tools directly. Breach, RTAMT, and STLCG++ are run in their native environments on a shared case set, and their per-timestep robustness outputs are compared at tolerance 10^{-6}. Table 2 summarizes the result: the baseline block contains 16 operator-coverage cases from the regression suite, while the divergence block isolates one implicit choice in each of five further cases.

The tools disagree even on small formulas and traces. In the baseline block, one tool deviates from the other two on 9 of the 16 cases; most disagreements come from Breach's terminal-step convention or RTAMT's boundary rule for windows extending past the trace end. As a concrete instance of the latter, for $\varphi = \Diamond_{[2,4]}(x \geq 0)$ evaluated at $t = 1$ on a trace ending at $T = 2$ with $x(T) = 3$, the window $[3, 5]$ lies entirely past the trace: clamp treats the last sample as persisting and reports $+3$ (satisfied), while the pessimistic rule sees an empty window and reports $-\infty$ (violated)—same formula, same samples, opposite verdicts. The divergence block isolates further choices such as window alignment, signal model, and equality predicates. The point is not that one tool is correct and another is wrong, but that the same nominal input can induce different robustness behavior under different implicit choices.

Diagnosis by validated compatibility backends. The discrepancies themselves are established by running the original tools. To inspect per-node outputs and to name the responsible implicit semantic choice mechanically, we built compatibility backends for each tool; each reproduces its tool's column of Table 2 per timestep (59 of 63 case–tool cells agree on computed robustness and three more on coverage; the single remaining mismatch is deliberate and documented). On a diverging case, a localizer reports the lowest syntax-tree node whose outputs disagree: on the boundary-rule headline row it returns `WindowMax[2,4]` vs. `DiscreteWindowMax[2,4]`, naming the boundary rule from §2 through the diagnostic architecture of §3. Thus the compatibility backends both reproduce the observed outputs and expose the minimal divergent formula node and responsible backend operations.

[4] The artifact repository provides reproduction instructions (`extra/experiments/`) for all experiments; Appendix C records supplementary protocol and setup details.

Table 2. Cross-tool divergence matrix: real Breach 1.11.4 (B), RTAMT 0.3.5 (R), and STLCG++ 0.0.2 with exact min/max aggregation (S), compared on per-timestep robustness at absolute tolerance 10^{-6} over a trace of length $|\sigma|$. Per tool, =: matches the common behavior on the case; $*$: differs from the other two tools; –: cannot express the case (a coverage finding); †: documented special handling (RTAMT reads a non-uniform time grid as uniformly indexed samples). Each diverging row names the responsible implicit choice from Table 1.

Case	$\|\sigma\|$	B	R	S	Implicit choice
Baseline block (operator coverage)					
$x \geq 3$	5	=	=	=	
$\neg(x \geq 0)$	5	=	=	=	
$(x \geq 0) \wedge (y \geq 0)$	5	$*$	=	=	terminal-step rule
$(x \geq 0) \vee (y \geq 0)$	5	$*$	=	=	terminal-step rule
$((x \geq 0) \wedge (y \geq 0)) \vee (z \geq 0)$	5	$*$	=	=	terminal-step rule
$\Box_{[0,2]}(x \geq 0)$	5	=	=	=	
$\Box_{[1,2]}(x \geq 0)$	3	=	$*$	=	boundary rule
$\Diamond_{[0,2]}(x \geq 0)$	5	=	=	=	
$\Diamond_{[1,3]}(x \geq 0)$	7	=	$*$	=	boundary rule
$\Diamond_{[1,2]}(x \geq 0)$	3	=	$*$	=	boundary rule
$(x \geq 0)\, U_{[0,3]}\, (y \geq 0)$	5	=	$*$	=	open vs. closed endpoints
$(x \geq 0)\, U_{[1,2]}\, (y \geq 0)$	5	=	$*$	=	open vs. closed endpoints
$(x \geq 0)\, U_{[0,4]}\, (y \geq 0)$	5	=	=	=	
$(x \geq 0)\, U_{[1,2]}\, (y \geq 0)$	2	=	$*$	=	boundary rule
$x + y \geq 0$	4	=	=	=	
$x - y \geq 0$	4	=	=	=	
Divergence block (one case per implicit choice)					
$\Diamond_{[2,4]}(x \geq 0)$	3	=	$*$	=	boundary rule
$(x \geq 0) \wedge (y \geq 0)$	3	$*$	=	=	terminal-step rule
$\Box_{[0.5,1.5]}(x > 0)$	3	=	–	–	window alignment (coverage)
$\Box_{[0,1]}(x > 0)$	5	=	†	–	signal model
$x = 3$	5	$*$	=	=	equality predicates

Consequences beyond robustness values. The discrepancies are not knife-edge coincidences of single constructed traces. Controlled trace sweeps show that the headline cases flip the verdicts a monitor reports and the trace rankings a search acts on over intervals of input values, not only at isolated points.

The same effect can compound downstream, as a purpose-built falsification task illustrates. We hold the system, specification, search algorithm, seeds, and budget fixed and vary only the objective semantics across three boundary/terminal-step rule combinations: clamp with the terminal step as-is, clamp with the extend-penultimate step, and pessimistic with as-is. The task is constructed so that the specification's monitor horizon reaches past the end of the trace, exactly where the boundary convention drives the objective. Figure 3 shows what happens in this case: the clamping variants falsify in 50/50 and 49/50 runs, while the pessimistic boundary rule collapses the objective to the same out-of-range value for every input and falsifies in only 10/50 runs.

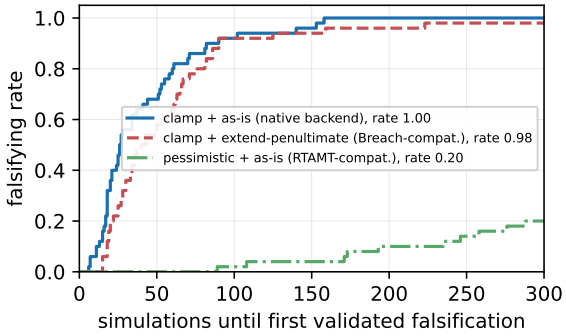

Fig. 3. Falsification performance under three objective semantics (named boundary/terminal-step rule combinations, realized as tidySTL backends), with the system, specification, search algorithm, seeds, and budget held fixed: ECDF, over 50 seeded runs, of the number of simulations until the first validated falsification (budget 300); the plateau height is the falsifying rate. Candidate violations are validated by an exact system-level check, so no backend acts as the verdict oracle.

Table 3. Implemented variations. The key column is the layer changed; LOC is reported only to make the locality claim auditable, not as a quality metric. The full regression suite passes with all variants installed.

Variant	Layer changed	LOC	Tests
Semantic variants (what is computed)			
Breach-compatible	executor op overrides	81	53
RTAMT-compatible	own lowering + executor	191	20
STLCG++-compatible	own lowering + executor	192	20
Computational variants (how it is computed)			
STLCG++ Torch executor	executor only	241	3
Rust SIMD executor	executor only (native ext.)	109 + 347 Rust	26

This is not evidence that one boundary rule is globally inferior. The collapse is a property of the pairing: a pessimistic boundary rule under a whole-trace monitor objective. A user who knows this convention is in play can restrict the monitor horizon and avoid the collapse. That is precisely the point: compensating for an implicit choice requires knowing that the choice is there, and nothing in the optimizer's output reveals it.

4.2 Implementing Evaluation Variations with Localized Changes

The previous subsection showed that implicit choices have observable consequences. We now measure the implementation cost of realizing such choices in tidySTL: how much code changes below the stable interface.

Implementation effort. Table 3 shows that the shipped variants change only backend-side components. Semantic variants change primitive operations, lowering rules, or executors below the stable core; computational variants keep the semantics fixed and change only execution. Thus locality here means a boundary of modification: the parser, formula representation, signal layer, and user-facing interface are reused unchanged.

5 Related Work

§2 defines the semantic scope of this paper. tidySTL differs from Breach, S-TaLiRo, RTAMT, and similar systems not by replacing their workflows but by making the evaluator layer itself the primary object of extension and comparison [6,7,1,18,21]. A supplementary comparison of representative tool characteristics is provided in Appendix B.

Proof-oriented monitoring work [5,3] makes correctness guarantees for monitoring algorithms a central object. tidySTL does not claim comparable formal verification; instead it makes the implicit choices explicit so that divergence sources can be identified without proof.

Speaking of implicitness is not a claim that tools such as RTAMT or Breach are poorly documented. The choices in question are simply not determined by the STL formula: adopting RTAMT, for instance, commits an evaluation to its discrete-time signal and pessimistic boundary rule regardless of the formula. The ARCH-COMP 2021 report [9] documented cross-tool discrepancies arising from exactly these implicit choices and resolved them by post-hoc independent checking; tidySTL instead makes the responsible choices explicit and switchable in advance.

MoonLight [17] shows that pluggable monitoring domains can be a first-class contribution; tidySTL similarly makes the implicit choices of STL robustness evaluation first-class. TeSSLa, RTLola, and HLola [13,11,12] demonstrate that front-end/back-end separation is itself a scientific contribution; their backends vary in execution platform while tidySTL's vary in the implicit choices they realize. STLCG and related differentiable tools [15,14] occupy the smooth-semantics niche that the STLCG++-compatible backend in tidySTL targets for comparison.

6 Conclusion and Future Work

tidySTL makes the implicit choices of STL robustness evaluation explicit at the evaluator boundary. By separating formula syntax, signal data, backend-specific lowering, and DAG interpretation, it exposes choices that are otherwise buried in tool internals and shows that they materially affect acceptance, verdicts, trace rankings, and falsification behavior across real, widely used tools—and that the choice responsible for a disagreement can be named mechanically.

The implementation supports the view of robustness evaluation as an extensible semantic layer rather than an implementation detail of a particular monitor,

falsifier, or learning pipeline. Future work will extend this layer to broader semantics, including online evaluation, and expand its validation suite.

A Scope and Implicit Semantic Choices in Detail

This appendix expands the implicit choices of Table 1 and organizes the sources of cross-tool difference. We first recall the reference robustness semantics, then detail finite-trace interpretation, coverage, and implementation-specific behavior. We finally distinguish explicit selection of advanced semantic extensions from implicit tool behavior around them.

A.1 Standard Robustness

Following Donzé and Maler [8], fix a signal σ in the sense of Definition 1 and a formula $\varphi \in \mathbf{Fml}$ as in Definition 2. The standard quantitative (robustness) semantics $\rho(\varphi, \sigma, t) \in \mathbb{R} \cup \{\pm\infty\}$ is defined inductively:

$$
\begin{aligned}
\rho(\top, \sigma, t) &= +\infty \\
\rho(f(\boldsymbol{x}) \geq 0, \sigma, t) &= f(\sigma_{\boldsymbol{x}}(t)) \\
\rho(\neg\varphi, \sigma, t) &= -\rho(\varphi, \sigma, t) \\
\rho(\varphi_1 \vee \varphi_2, \sigma, t) &= \max(\rho(\varphi_1, \sigma, t),\ \rho(\varphi_2, \sigma, t)) \\
\rho(\varphi_1 \wedge \varphi_2, \sigma, t) &= \min(\rho(\varphi_1, \sigma, t),\ \rho(\varphi_2, \sigma, t)) \\
\rho(\varphi_1\, U_I\, \varphi_2, \sigma, t) &= \sup_{t' \in t+I} \min\Big(\rho(\varphi_2, \sigma, t'),\ \inf_{t'' \in [t,\, t')} \rho(\varphi_1, \sigma, t'')\Big) \\
\rho(\Diamond_I \varphi, \sigma, t) &= \sup_{t' \in t+I} \rho(\varphi, \sigma, t') \\
\rho(\Box_I \varphi, \sigma, t) &= \inf_{t' \in t+I} \rho(\varphi, \sigma, t'),
\end{aligned}
$$

where $t+I = \{t+s \mid s \in I\}$ and $f(\sigma_{\boldsymbol{x}}(t))$ applies f to the values of the variables $\boldsymbol{x}$ at time t. The release operator is defined by duality, $\varphi_1\, R_I\, \varphi_2 :\equiv \neg(\neg\varphi_1\, U_I\, \neg\varphi_2)$, and the constant $\bot :\equiv \neg\top$ has $\rho(\bot, \sigma, t) = -\infty$.

Read literally, this definition assumes an idealized signal: continuous, infinitely long, and known at every instant. An evaluator never has that. It has a finite list of samples $\sigma(t_0), \ldots, \sigma(t_n)$, and to compute the sup, inf, and min above it must decide what the signal does at the times that fall between, before, and after those samples. Those decisions form the finite-trace interpretation discussed in Appendix A.3.

A.2 Coverage and Implementation Deviations

Support for until and release. Few tools implement the full syntax of Definition 2. The until operator, with its nested inf, is harder to implement and to vectorize than $\Box$ and $\Diamond$, and release is often dropped together with it: some tools support

only the box/diamond fragment, others support until under restricted interval forms. The effect is on coverage: a specification accepted by one tool is rejected by another before any robustness is computed. This is a syntax-coverage choice: adopting the tool implicitly selects which formulas can be evaluated.

Range of expressible atomic predicates. Definition 2 permits an arbitrary function symbol f, but front ends restrict what is expressible: to threshold comparisons on single variables, to linear arithmetic, or to a fixed expression grammar. This is also a coverage choice.

Open vs. closed interval endpoints. Definition 2 admits open and half-open intervals such as $(1, 3]$, but many front ends parse only closed ones. Where open intervals are accepted, their reading depends on the signal interpretation: under a continuous interpolant the inf over an open window coincides with that over its closure, so the difference is invisible, while under a discrete reading excluding an endpoint drops a sample and can change the aggregated value. Rejecting an open interval is a coverage choice; silently treating it as closed is an implementation deviation. Both are implicit because the formula does not announce how the evaluator will handle it.

Robustness of $\top$ and $\bot$. The standard semantics assigns $\rho(\top, \sigma, t) = +\infty$ and $\rho(\bot, \sigma, t) = -\infty$. Tools differ in whether they admit infinite values at all: some propagate genuine infinities through the min/max aggregation, others substitute large finite sentinels, and a sentinel leaks into reported values once it is aggregated, scaled, or differentiated. Since the reference semantics fixes genuine infinities, a finite sentinel is an implementation deviation. It is nevertheless an implicit semantic choice in our taxonomy because adopting the evaluator silently selects this effective behavior.

A.3 Implicit Semantic Choices for Finite Inputs

The most consequential category is the *finite-trace interpretation*: the bundle of implicit choices, summarized in Table 1, by which the finite timed state sequence an evaluator receives is read as the signal σ that the semantics quantifies over. We discuss its choices in turn, then the remaining implicit choices in Table 1.

Signal model. The signal model decides how a sampled trace is read as a function of continuous time. Under a *piecewise-linear* dense-time model (Breach, and tidySTL's native backend) the value between two samples is their linear interpolant, so a predicate such as $x \geq 0$ can change truth between sample points. A *piecewise-constant* model holds each sample until the next. A *discrete-time* model (RTAMT's default) drops the notion of between-sample time altogether: the trace is an indexed sequence and temporal operators count indices, not durations. The same formula and the same samples can therefore denote different signals before evaluation even begins.

Window alignment. Even once the signal model is fixed, a temporal window $[t + a, t + b]$ has endpoints that need not coincide with samples. The window-alignment rule decides the value taken there. A dense-time backend evaluates the interpolant at the exact endpoint; a discrete backend snaps to the nearest index or ignores the fractional part. This choice is invisible on uniformly sampled traces whose windows align with samples, which is why it is easy to overlook, and why divergence appears only on alignment-sensitive inputs.

Boundary rule. When a window $[t + a, t + b]$ reaches past the end of the trace at T, the sup or inf ranges in part over times where no sample exists. The boundary rule supplies those values. Under *clamp* (tidySTL, Breach) the last sample is treated as persisting, so an out-of-range query returns $\sigma(T)$. Under a *pessimistic* rule (RTAMT discrete) the out-of-range part contributes nothing: the window is effectively empty, so a $\Diamond$ becomes $-\infty$ (the sup of an empty set) and a $\Box$ becomes $+\infty$. This is the boundary-rule row of the divergence block in Table 2. For $\varphi = \Diamond_{[2,4]}(x \geq 0)$ evaluated at $t = 1$ on a trace that ends at $T = 2$ with $x(T) = 3$, the window $[3, 5]$ lies entirely beyond the trace. Clamp sees the persisting value 3 and reports $+3$ (satisfied); the pessimistic rule sees an empty window and reports $-\infty$ (violated). Same formula, same samples, same time point; opposite verdicts.

Terminal-step rule. At the final sample t_N a tool may report the robustness it computed there, or it may overwrite it with the value from the penultimate sample t_{N-1}. Breach does the latter by default; RTAMT does the former. The two agree whenever the final and penultimate robustness coincide, so the choice is silent on most traces. It is not silent when they differ. This is the terminal-step row of the divergence block in Table 2. For $\varphi = (x \geq 0) \wedge (y \geq 0)$ on trace A with $x = y = [5, 5, -3]$, the penultimate robustness is $+5$ and the final is -3. The standard rule reports -3; the extension rule copies the penultimate $+5$. Against a trace B that holds $+0.1$ throughout, this flips which trace looks better, reversing a ranking that a falsification search would act on.

Robustness of atomic predicates. Even for an accepted predicate, the evaluator must decide how it is scored: the robustness of $f(\boldsymbol{x}) \geq 0$ may be the raw signed margin $f(\boldsymbol{x})$ or a Euclidean distance to the satisfaction set [10], and the two disagree once a predicate involves several variables.

A.4 Advanced Semantic Extensions

Applications genuinely demand more than Definition 2 provides. The following extensions explicitly select a different language or evaluation problem. That selection is not implicit when exposed to the user; whether a tool supports the extension is an implicit coverage choice, and any tool-fixed behavior within the selected extension remains an implicit choice.

Equality predicates. The equality $x = 0$ is absent from Definition 2 yet common in practical specifications. Adding equality to the language is an explicit extension; once a tool accepts it, however, the assigned robustness is an implicit choice because the formula does not select one. Equality is satisfied only at the single value $x = 0$ and admits no single standard robustness; a common choice is $-|x|$, which is never positive, so even a satisfied equality reports non-positive robustness. Breach instead scores equality by a fixed-magnitude sign convention ($\pm 10^4$), so the reported value carries no metric information at all—the equality row of Table 2 shows the resulting cross-tool divergence on inputs every tool accepts.

Past-time operators. Operators such as once and historically mirror the temporal modalities into the past. They change the direction in which an evaluator walks the trace, and tools either provide them (RTAMT) or not. The effect is on coverage.

Differentiable robustness. Gradient-based learning replaces the min/max aggregations by smooth approximations [15,19]. The smooth value deviates from the standard robustness by construction. Whether the deviation is exposed as a tunable parameter or buried in the tool determines its classification: explicitly selecting smooth robustness changes the evaluation problem, while a buried approximation is implicit implementation behavior.

Online evaluation. Online monitoring evaluates a growing prefix, so verdicts must be issued before the trace is complete [4]. This forces a treatment of missing future information—three-valued verdicts, robustness intervals—that offline evaluation never confronts.

A.5 Why "Implicit"

The choices in Table 1 are implicit because they are absent from both the STL formula and the finite sampled input, yet are fixed by adopting the evaluator. Choosing a tool therefore silently chooses a combination of finite-trace conventions, scoring rules, coverage boundaries, and implementation behavior. "Choice" here names that effective behavior without judging whether it is deliberate. Two tools whose combinations differ can reject different inputs, report different robustness, or even produce opposite verdicts on identical input. The contribution of tidySTL is not to declare one combination correct, but to expose and diagnose the combination rather than leave it as a property buried in the tool.

B The Tool Landscape in Detail

Table 4 compares representative tools along the axes on which their evaluators actually differ, expanding the summary in §2.

Table 4. How representative tools differ along the axes that shape their evaluators.

Tool	Interface	Signal model	Online mode	Primary target	Auto-grad	Stack
Breach [6,7]	API + DSL	piecewise-linear	offline	falsification	no	MATLAB, mature
RTAMT [18,21]	DSL + API	discrete (default) / dense	delayed	runtime verification	no	Python, heavy
MoonLight [17]	DSL + API	spatial + temporal	partial	spatial RV	no	Java / CLI / Python
STLCG++ [14]	API	discrete (tensor)	none	differentiable learning	yes	PyTorch

C Experimental Details

This appendix records the protocols and full data behind the findings reported in §4.

Artifact and reproducibility. The public artifact repository contains the implementation, fixtures, and reference-tool adapters. The user manual documents installation, backend semantics, and worked examples; the experiment guide documents how to regenerate every evaluation record, table, and figure.

Case set and comparison protocol. The baseline block comprises 16 operator-coverage cases drawn from the regression suite; the divergence block adds five cases, each targeting one implicit choice from Table 1. Each tool runs in its native environment (versions as in Table 2). Outputs are normalized to per-timestep robustness vectors and compared pairwise at absolute tolerance 10^{-6} on mutually defined timesteps. STLCG++ runs with exact min/max aggregation—its default configuration; smooth aggregation is opt-in—so its column reflects its discrete-time semantics rather than smoothing, which remains an advanced semantic extension not exercised here.

Compatibility validation. Every tool column of Table 2 doubles as a validation check: on each case, the named compatibility backend must reproduce its tool's output per timestep at 10^{-6}—21 cases $\times$ 3 tools, hence 63 case–tool cells. The check passes on all 59 cells that carry values; three further cells match on coverage (backend and tool agree the case is inexpressible). The one mismatch is deliberate: where real RTAMT silently reinterprets a non-uniform grid as uniformly indexed samples, the RTAMT-compatible backend rejects the input instead of reproducing the reinterpretation. The backends implement each tool's *documented* rules; the reproduction check verifies the implementations rather than defining them—and it is nontrivial: on the first live Breach run it exposed two fidelity bugs in our own Breach-compatible backend (the equality convention and the window-endpoint reduction), both fixed and re-verified before the numbers reported here were produced.

Localization procedure. On the validated columns, the localizer exploits that tidySTL backends share one formula representation with inspectable per-node outputs: it walks the shared syntax tree in post-order and reports the *minimal divergent node*—the lowest node whose outputs disagree while all its children agree—together with each backend's primitive operation at that node. On the two headline rows of Table 2:

```
first divergent op at ‘F[2,4]‘: WindowMax[2,4] vs
    DiscreteWindowMax[2,4] (t-index 1) -> boundary rule
first divergent op at ‘and‘: PointwiseMin vs PointwiseMin
    + terminal-step extension (t-index 2) -> terminal-step rule
```

Backend pairs whose lowerings differ structurally are compared at the shared syntax-tree level rather than between lowered representations; predicate-internal arithmetic is treated as a leaf, since none of the implicit choices studied here concerns its internal operations.

Trace-family construction. A single constructed trace could sit on a knife edge, so each headline case is widened to a swept family of 201 traces whose final sample varies over $[-5, 5]$, spanning regions of agreement and disagreement; no rule is taken as the reference, and the rates are symmetric between the rules by construction. For the boundary-rule family ($\Diamond_{[2,4]}(x \geq 0)$, window past the trace end), the clamping and pessimistic rules return opposite verdicts at the divergence timestep on exactly those traces whose final sample is non-negative (101 of 201). The terminal-step family is a conjunction violated only at the final sample; the as-is and extend-penultimate rules flip the final verdict on 100 of 201 traces and rank the trace oppositely against a fixed reference trace with an interior violation—the ranking signal a falsification search acts on—on 80 of 201. These rates measure the width of the disagreement region within each controlled family—the divergences are not measure-zero coincidences—not a frequency claim about any particular workload.

Falsification setup. The system is a closed-form speed model $\dot{v} = Ku(t) - c\,v$ with piecewise-constant throttle on four segments (a bounded $[0, 1]^4$ search space; traces sampled exactly, no integrator); the specification is $(v \leq 8.8) \wedge \Diamond_{[0.5,1.5]}(v \leq 25)$, whose second conjunct never fails but carries the monitor horizon past the end of the trace. A fixed (1+1)-ES with random restarts minimizes the monitor-style objective $\min_t \rho(t)$; only the objective semantics varies across arms, and every claimed falsification is validated by an exact closed-form check of the system itself, so no backend acts as the verdict oracle. The configuration, seeds, and budget (50 seeded runs, 300 simulations each) were frozen before the runs.

Per-arm results behind Figure 3: under the clamping boundary rule the search falsifies in 50/50 runs (median 26.5 simulations); adding the terminal-step extension masks violations that appear only at the final sample, costing one run and roughly 1.5× the simulations (49/50, median 40). Under the pessimistic rule the out-of-range window makes the objective $-\infty$ for every input, so the search degenerates to blind sampling: 10/50 runs falsify (median 214), and 14,027 candidate falsifications claimed across the campaign are rejected by the exact system-level check. The search runs to completion in every arm; the disagreement between the arms is silent.

Variant accounting. LOC in Table 3 counts variant-specific code only: blank, comment-only, and docstring lines are excluded, shared infrastructure is excluded, and Rust is counted separately. The native backend is the baseline and has no row; the Rust SIMD executor ships as an optional native extension alongside the five backends. The full regression suite comprises 181 tests and passes with all variants installed.

Performance sanity check. The benchmark in Figure 4 compares tidySTL backends with real RTAMT and STLCG++ on the same workload. On single traces, the explicit evaluator layer remains within the same order of magnitude as the Python-stack references. On batches, tidySTL's batch-first signal layout amortizes per-trace cost, making the native backend substantially faster than looping RTAMT and faster than the tensor-batched STLCG++ run shown here. The SIMD executor gives the expected speed ceiling once per-node observability is dropped. This check shows that inspectability does not impose prohibitive overhead; performance is not a primary evaluation claim.

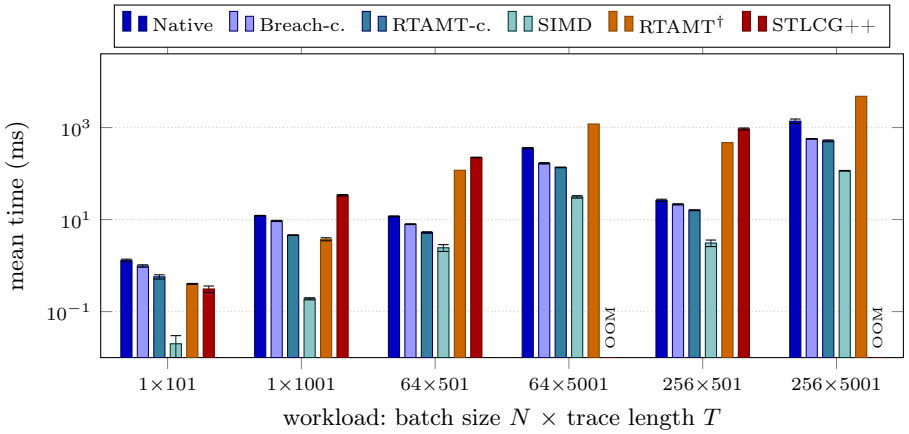

Fig. 4. Mean evaluation time on representative workloads, plotted on a log scale. All bars use the same formula $\square_{[0,5]}((x > 0) \wedge \Diamond_{[0,2]}(y > 0))$; error bars show one standard deviation. RTAMT† batch times are $N\times$ the measured per-trace time; missing STLCG++ bars exceed memory (OOM).

Table 5 tabulates the measurements plotted in Figure 4.

References

1. Annpureddy, Y., Liu, C., Fainekos, G.E., Sankaranarayanan, S.: S-TaLiRo: A tool for temporal logic falsification for hybrid systems. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS). Lecture Notes in Computer Science, vol. 6605, pp. 254–257. Springer (2011). https://doi.org/10.1007/978-3-642-19835-9_21

Table 5. Mean evaluation time (ms) ± std over a batch of N traces of length T; identical workload $\square_{[0,5]}((x > 0) \wedge \Diamond_{[0,2]}(y > 0))$ for every column. This is the data plotted in Figure 4. †: RTAMT evaluates one trace per call, so batch times are $N\times$ the measured per-trace time (linearity checked at $N \in \{1, 8\}$). OOM: window materialization exceeds memory on the 32 GB host.

N	T	Native	Breach-c.	RTAMT-c.	SIMD	RTAMT	STLCG++
1	101	1.30 ± 0.08	0.98 ± 0.06	0.57 ± 0.06	0.02 ± 0.01	0.40 ± 0.01	0.31 ± 0.05
1	1001	12.15 ± 0.19	9.34 ± 0.28	4.58 ± 0.08	0.19 ± 0.01	3.74 ± 0.28	33.71 ± 1.31
64	501	11.75 ± 0.25	7.94 ± 0.15	5.24 ± 0.20	2.44 ± 0.41	$118^{\dagger}$	224 ± 3
64	5001	358 ± 11	167 ± 6	136 ± 2	31.20 ± 1.89	$1195^{\dagger}$	OOM
256	501	26.19 ± 1.54	21.42 ± 0.65	15.97 ± 0.37	3.09 ± 0.50	$471^{\dagger}$	934 ± 44
256	5001	1377 ± 155	569 ± 9	515 ± 23	115 ± 2	$4781^{\dagger}$	OOM

2. Bartocci, E., Deshmukh, J., Donzé, A., Fainekos, G., Maler, O., Ničković, D., Sankaranarayanan, S.: Specification-based monitoring of cyber-physical systems: A survey on theory, tools and applications. In: Lectures on Runtime Verification: Introductory and Advanced Topics, Lecture Notes in Computer Science, vol. 10457, pp. 135–175. Springer (2018). https://doi.org/10.1007/978-3-319-75632-5_5
3. Chattopadhyay, A., Mamouras, K.: A verified online monitor for metric temporal logic with quantitative semantics. In: Runtime Verification (RV). Lecture Notes in Computer Science, vol. 12399, pp. 383–403. Springer (2020). https://doi.org/10.1007/978-3-030-60508-7_21
4. Deshmukh, J.V., Donzé, A., Ghosh, S., Jin, X., Juniwal, G., Seshia, S.A.: Robust online monitoring of signal temporal logic. Formal Methods in System Design **51**(1), 5–30 (2017). https://doi.org/10.1007/s10703-017-0286-7
5. Dokhanchi, A., Hoxha, B., Fainekos, G.E.: On-line monitoring for temporal logic robustness. In: Runtime Verification (RV). Lecture Notes in Computer Science, vol. 8734, pp. 231–246. Springer (2014). https://doi.org/10.1007/978-3-319-11164-3_19
6. Donzé, A.: Breach, a toolbox for verification and parameter synthesis of hybrid systems. In: Computer Aided Verification (CAV). Lecture Notes in Computer Science, vol. 6174, pp. 167–170. Springer (2010). https://doi.org/10.1007/978-3-642-14295-6_17
7. Donzé, A., Ferrère, T., Maler, O.: Efficient robust monitoring for STL. In: Computer Aided Verification (CAV). Lecture Notes in Computer Science, vol. 8044, pp. 264–279. Springer (2013). https://doi.org/10.1007/978-3-642-39799-8_19
8. Donzé, A., Maler, O.: Robust satisfaction of temporal logic over real-valued signals. In: Formal Modeling and Analysis of Timed Systems (FORMATS). Lecture Notes in Computer Science, vol. 6246, pp. 92–106. Springer (2010). https://doi.org/10.1007/978-3-642-15297-9_9
9. Ernst, G., Arcaini, P., Bennani, I., Chandratre, A., Donzé, A., Fainekos, G., Frehse, G., Gaaloul, K., Inoue, J., Khandait, T., Mathesen, L., Menghi, C., Pedrielli, G., Pouzet, M., Waga, M., Yaghoubi, S., Yamagata, Y., Zhang, Z.: ARCH-COMP 2021 category report: Falsification with validation of results. In: 8th International Workshop on Applied Verification of Continuous and Hybrid Systems (ARCH21). EPiC Series in Computing, vol. 80, pp. 133–152. EasyChair (2021). https://doi.org/10.29007/xwl1

10. Fainekos, G.E., Pappas, G.J.: Robustness of temporal logic specifications for continuous-time signals. Theoretical Computer Science **410**(42), 4262–4291 (2009). https://doi.org/10.1016/j.tcs.2009.06.021
11. Faymonville, P., Finkbeiner, B., Schledjewski, M., Schwenger, M., Stenger, M., Tentrup, L., Torfah, H.: StreamLAB: Stream-based monitoring of cyber-physical systems. In: Computer Aided Verification (CAV). Lecture Notes in Computer Science, vol. 11561, pp. 421–431. Springer (2019). https://doi.org/10.1007/978-3-030-25540-4_24
12. Gorostiaga, F., Sánchez, C.: HLola: A very functional tool for extensible stream runtime verification. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS). Lecture Notes in Computer Science, vol. 12652, pp. 349–356. Springer (2021). https://doi.org/10.1007/978-3-030-72013-1_18
13. Kallwies, H., Leucker, M., Schmitz, M., Schulz, A., Thoma, D., Weiss, A.: TeSSLa – an ecosystem for runtime verification. In: Runtime Verification (RV). Lecture Notes in Computer Science, vol. 13498, pp. 314–324. Springer (2022). https://doi.org/10.1007/978-3-031-17196-3_20
14. Kapoor, P., Mizuta, K., Kang, E., Leung, K.: STLCG++: A masking approach for differentiable signal temporal logic specification. IEEE Robotics and Automation Letters **10**(9), 9240–9247 (2025). https://doi.org/10.1109/LRA.2025.3588389
15. Leung, K., Aréchiga, N., Pavone, M.: Back-propagation through signal temporal logic specifications: Infusing logical structure into gradient-based methods. In: Algorithmic Foundations of Robotics XIV (WAFR). Springer Proceedings in Advanced Robotics, vol. 17, pp. 432–449. Springer (2021). https://doi.org/10.1007/978-3-030-66723-8_26
16. Maler, O., Nickovic, D.: Monitoring temporal properties of continuous signals. In: Formal Techniques, Modelling and Analysis of Timed and Fault-Tolerant Systems (FORMATS/FTRTFT). Lecture Notes in Computer Science, vol. 3253, pp. 152–166. Springer (2004). https://doi.org/10.1007/978-3-540-30206-3_12
17. Nenzi, L., Bartocci, E., Bortolussi, L., Silvetti, S., Loreti, M.: MoonLight: A lightweight tool for monitoring spatio-temporal properties. International Journal on Software Tools for Technology Transfer **25**(4), 503–517 (2023). https://doi.org/10.1007/s10009-023-00710-5
18. Nickovic, D., Yamaguchi, T.: RTAMT: Online robustness monitors from STL. In: Automated Technology for Verification and Analysis (ATVA). Lecture Notes in Computer Science, vol. 12302, pp. 564–571. Springer (2020). https://doi.org/10.1007/978-3-030-59152-6_34
19. Pant, Y.V., Abbas, H., Mangharam, R.: Smooth operator: Control using the smooth robustness of temporal logic. In: IEEE Conference on Control Technology and Applications (CCTA). pp. 1235–1240. IEEE (2017). https://doi.org/10.1109/CCTA.2017.8062628
20. Raman, V., Donzé, A., Maasoumy, M., Murray, R.M., Sangiovanni-Vincentelli, A., Seshia, S.A.: Model predictive control with signal temporal logic specifications. In: 53rd IEEE Conference on Decision and Control (CDC). pp. 81–87. IEEE (2014). https://doi.org/10.1109/CDC.2014.7039363
21. Yamaguchi, T., Hoxha, B., Nickovic, D.: RTAMT – runtime robustness monitors with application to CPS and robotics. International Journal on Software Tools for Technology Transfer **26**(1), 79–99 (2024). https://doi.org/10.1007/s10009-023-00720-3